Задания №19-21, программирование | Часть 2

Уровень: "Нет данных"

Задание №1

Полина и Вероника играют в игру, правила которой заключаются в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в одну кучу 2 камня или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается в тот момент, когда сумма камней двух куч стала не менее 89.
4) Победителем является тот игрок, который сделал последний ход и получил сумму, равную 89 или больше.
Пример: имея кучи (10, 5) камней, за один ход игрок может получить: (12, 5), (30, 5), (10, 7), (10, 15).
В начальный момент в кучах было (10, S) камней, где S в диапазоне [1, 78].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность выиграть, набрав нужную сумму камней в куче при любых ходах противника.
Определите и запишите в ответ минимальное значение S, при котором Полина может выиграть за один ход.

Верные варианты

27

INSANEK.RU

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 89 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за один ход, то выведем S.
Код:

```
def g(x, s, p, end):
  if (x + s) >= 89 and p in end:
    return True
  if (x + s) < 89 and p == max(end):
    return False
  if (x + s) >= 89 and p not in end:
    return False
  if (p + 1) % 2 == (end[0] % 2):
    return (g(x + 2, s, p + 1, end) or g(x * 3, s, p + 1, end) or g(x, s + 2, p + 1, end) or g(x, s * 3, p + 1, end))
  else:
    return (g(x + 2, s, p + 1, end) and g(x * 3, s, p + 1, end) and g(x, s + 2, p + 1, end) and g(x, s * 3, p + 1, end))

x = 10
for s in range(1, 79):
  if g(x, s, 0, [1]):
    print(s)
    break
```

Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=89) количество камней в двух кучах:

```
def Win1(x, S):
  return (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89))
```

2. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:

```
x = 10
S19 = []
```

3. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

```
for S in range(1, 79):
```

4. Делаем проверку:

```
if Win1(x, S):
```

Она работает так: если при данной позиции Полина сможет выиграть за 1 ход, то такая позиция подходит и мы добавим её в список:

```
S19.append(S)
```

5. Выводим на экран минимальное значение из списка — это и будет ответом:
print(min(S19))

Полный код:
```
def Win1(x, S):
   return (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89))
x = 10
S19 = []
for S in range(1, 79):
   if Win1(x, S):
      S19.append(S)
print(min(S19))
```

Задание №2

Полина и Вероника играют в игру, правила которой заключаются в следующем:
1) Перед игроками лежат две кучи камней. Игроки ходят по очереди, первый ход делает Полина.
2) За один ход игрок может добавить в одну кучу 2 камня или увеличить количество камней в куче в 3 раза. Для этого у каждого игрока есть неограниченный запас камней.
3) Игра завершается в тот момент, когда сумма камней двух куч стала не менее 89.
4) Победителем является тот игрок, который сделал последний ход и получил сумму, равную 89 или больше.
Пример: имея кучи (10, 5) камней, за один ход игрок может получить: (12, 5), (30, 5), (10, 7), (10, 15).
В начальный момент в кучах было (10, S) камней, где S в диапазоне [1, 78].
Примечание. Игрок обладает выигрышной стратегией, если у него есть возможность выиграть, набрав нужную сумму камней в куче при любых ходах противника.
Найдите значение S, при котором параллельно выполняются два условия:
— у Вероники есть стратегия, позволяющая ей выиграть первым или вторым ходом при любой игре Полины;
— у Вероники нет стратегии, которая позволит ей гарантированно выиграть первым ходом.

INSANEK.RU

Верные варианты

23

Решение

Сообщить об ошибке

Решение программированием. Способ 1:
1. Объявляем функцию g(s, p, end), которая будет вычислять результат игры, где s — текущее количество камней в куче, p — текущий ход игры, end — количество ходов, за которое должна завершиться игра.
2. Внутри функции пропишем, что если сумма камней в кучах не менее 89 и игразавершилась за необходимое количество ходов, то вернем истину, в остальных случаях —ложь.
3. Если следующий ход будет такой же четности, что и последний, то нам нужна хотя бы одним ходом, иначе — всеми вариантами ходов.
4. Запустим цикл для перебора значений S. Если при текущем значении игра может завершиться за два или за четыре хода, то выведем S.
Код:
```
def g(x, s, p, end):
   if (x + s) >= 89 and p in end:
      return True
   if (x + s) < 89 and p == max(end):
      return False
   if (x + s) >= 89 and p not in end:
      return False
   if (p + 1) % 2 == (end[0] % 2):
      return (g(x + 2, s, p + 1, end) or g(x * 3, s, p + 1, end) or g(x, s + 2, p + 1, end) or g(x, s * 3, p + 1, end))
   else:
      return (g(x + 2, s, p + 1, end) and g(x * 3, s, p + 1, end) and g(x, s + 2, p + 1, end) and g(x, s * 3, p + 1, end))

x = 10
for s in range(1, 79):
   if g(x, s, 0, [2, 4]) and not(g(x, s, 0, [2])):
      print(s)
```
Решение программированием. Способ 2:
1. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за один ход. Назовём её Win1, а в качестве параметра будем передавать количество камней в первой куче и возможное начальное значение S.
Функция будет возвращать результат выражения (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89)). То есть результатом будет True (если позиция подходит) или False (если позиция не подходит). Это помогает узнать, можно ли за один ход получить победное (>=89) количество камней в двух кучах:
```
def Win1(x, S):
   return (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89))
```
2. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 ход. Проигрышная за 1 ход позиция — это

позиция, из которой соперник 100% выиграет, как бы мы ни сходили (причём выиграет своим первым ходом).
Функция будет возвращать результат выражения ((not(Win1(x, S))) and Win1(x + 2, S) and Win1(x, S + 2) and Win1(x * 3, S) and Win1(x, S * 3)).
Что должно выполняться, чтобы функция вернула True?
1) Текущая позиция не должна быть выигрышной за 1 ход (not(Win1(x, S))).
2) Как бы ни сходил игрок, у него должна получаться такая позиция, из которой соперник победит своим первым ходом.

def Loss1(x, S):
return ((not(Win1(x, S))) and Win1(x + 2, S) and Win1(x, S + 2) and Win1(x * 3, S) and Win1(x, S * 3))

3. Создаём функцию, которая будет проверять, является ли текущая позиция выигрышной за два хода. То есть может ли игрок, который будет из неё ходить, победить своим вторым ходом.
Функция будет возвращать результат выражения (Loss1(x + 2, S) or Loss1(x, S + 2) or Loss1(x * 3, S) or Loss1(x, S * 3)). Что должно выполняться, чтобы функция вернула True?
У игрока, который делает ход из этой позиции, должен быть хотя бы один вариант, который сможет принести проигрыш сопернику. То есть один из возможных ходов должен создать проигрышную за 1 ход позицию (поэтому пишем or), чтобы соперник не смог победить, а сам игрок своим вторым ходом одержал победу.

def Win2(x, S):
return (Loss1(x + 2, S) or Loss1(x, S + 2) or Loss1(x * 3, S) or Loss1(x, S * 3))

4. Создаём функцию, которая будет проверять, является ли текущая позиция проигрышной за 1 или 2 хода (Вероника выигрывает за 1 или 2 хода, следовательно, Полина проигрывает за 1 или 2 хода).
Функция будет возвращать True или False в зависимости от значения выражения. Истина возможна при выполнении условия: позиции, которые можно получить из текущей, 100% выигрышные — либо за 1 ход, либо за 2.
Чтобы исключить вариант, что все позиции, которые можно получить из (x, S), выигрышные только за 1 ход, добавим условие (Win2(x + 2, S) or Win2(x, S + 2) or Win2(x * 3, S) or Win2(x, S * 3)). Оно обеспечит хотя бы один вариант победы за 2 хода.

def Loss12(x, S):
return ((Win1(x + 2, S) or Win2(x + 2, S)) and (Win1(x, S + 2) or Win2(x, S + 2)) and (Win1(x * 3, S) or Win2(x * 3, S)) and (Win1(x, S * 3) or Win2(x, S * 3)) and (Win2(x + 2, S) or Win2(x, S + 2) or Win2(x * 3, S) or Win2(x, S * 3)))

INSANEK.RU

5. Создаём список, в котором будут храниться все подходящие под условие значения S, и переменную, в которой будет храниться количество камней в первой куче:

x = 10
S21 = []

6. Создаём цикл для перебора возможных значений S. Диапазон перебора указан в условии задачи:

for S in range(1, 79):

7. Делаем проверку:

if Loss12(x, S):

Она работает так: если из текущей позиции можно проиграть первым или вторым ходом, то такая позиция подходит и мы добавим её в список:

S21.append(S)

8. Выводим на экран весь список и выбираем ответ:

print(S21)

Полный код:

```
def Win1(x, S):
   return (x + S < 89) and ((x + 2 + S >= 89) or (x * 3 + S >= 89) or (S * 3 + x >= 89))
def Loss1(x, S):
   return ((not(Win1(x, S))) and Win1(x + 2, S) and Win1(x, S + 2) and Win1(x * 3, S) and Win1(x, S * 3))
def Win2(x, S):
   return (Loss1(x + 2, S) or Loss1(x, S + 2) or Loss1(x * 3, S) or Loss1(x, S * 3))
def Loss12(x, S):
   return ((Win1(x + 2, S) or Win2(x + 2, S)) and (Win1(x, S + 2) or Win2(x, S + 2)) and (Win1(x * 3, S) or Win2(x * 3, S)) and (Win1(x, S * 3) or Win2(x, S *
3)) and (Win2(x + 2, S) or Win2(x, S + 2) or Win2(x * 3, S) or Win2(x, S * 3)))
x = 10
S21 = []
for S in range(1, 79):
   if Loss12(x, S):
      S21.append(S)
print(S21)
```